EPIC 06 — Task 03: Saga Pattern

The task is to **design a transaction** using the following flow shown in your screenshot:

Order
↓
Payment
↓
Inventory
↓
Confirmation

The goal is to implement **success, rollback, and compensation scenarios**.

Step 1 — Saga Flow

Customer
↓
Order Service
↓
Payment Service
↓
Inventory Service
↓
Confirmation Service

Each service performs its own local transaction.

Step 2 — Successful Transaction

1. Order Service

Create the order.

Order → CREATED

2. Payment Service

Process the payment.

Payment → SUCCESS

3. Inventory Service

Reserve the product.

Inventory → RESERVED

4. Confirmation

Confirm the order.

Order → CONFIRMED

Complete flow:

Order
↓
Payment ✓
↓
Inventory ✓
↓
Confirmation ✓

Step 3 — Failure Scenario

Suppose payment succeeds, but inventory reservation fails.

Order ✓
↓
Payment ✓
↓
Inventory ✗

The Saga must perform a **compensating transaction**.

Inventory fails
↓
Refund Payment
↓
Cancel Order

Final state:

Order → CANCELLED
Payment → REFUNDED
Inventory → NOT RESERVED

Step 4 — Saga Compensation

Step	Action	If successful	Compensation if failed later
1	Create Order	ORDER_CREATED	Cancel Order
2	Process Payment	PAYMENT_SUCCESS	Refund Payment
3	Reserve Inventory	INVENTORY_RESERVED	Release Inventory
4	Confirm Order	ORDER_CONFIRMED	Cancel Order

Step 5 — Implement Saga Events

For example, create these events:

OrderCreatedEvent
PaymentCompletedEvent
PaymentFailedEvent
InventoryReservedEvent
InventoryFailedEvent
OrderConfirmedEvent
OrderCancelledEvent
PaymentRefundedEvent

The services communicate using events rather than directly sharing databases.

Step 6 — Example Saga Code

Order Service

```
@Service
public class OrderService {

   public void createOrder(Order order) {

      // Save order
      order.setStatus("CREATED");
      orderRepository.save(order);

      // Publish event
      OrderCreatedEvent event =
            new OrderCreatedEvent(order.getId());

      kafkaTemplate.send("order-created", event);
   }
}
```

Payment Service

```
@KafkaListener(topics = "order-created")
public void processPayment(OrderCreatedEvent event) {

   try {
      // Process payment

     kafkaTemplate.send(
        "payment-completed",
        new PaymentCompletedEvent(event.getOrderId())
      );

   } catch (Exception e) {

      kafkaTemplate.send(
```

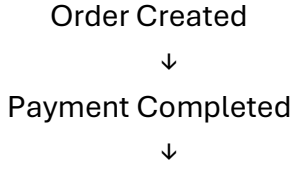
```
            "payment-failed",
            new PaymentFailedEvent(event.getOrderId())
        );
    }
}
```

Inventory Service

```
@KafkaListener(topics = "payment-completed")
public void reserveInventory(PaymentCompletedEvent event) {

    try {
        // Reserve inventory

        kafkaTemplate.send(
            "inventory-reserved",
            new InventoryReservedEvent(event.getOrderId())
        );

    } catch (Exception e) {

        kafkaTemplate.send(
            "inventory-failed",
            new InventoryFailedEvent(event.getOrderId())
        );
    }
}
```

Step 7 — Compensation Flow

If Inventory fails:



Inventory Failed
↓
Payment Refund
↓
Order Cancelled

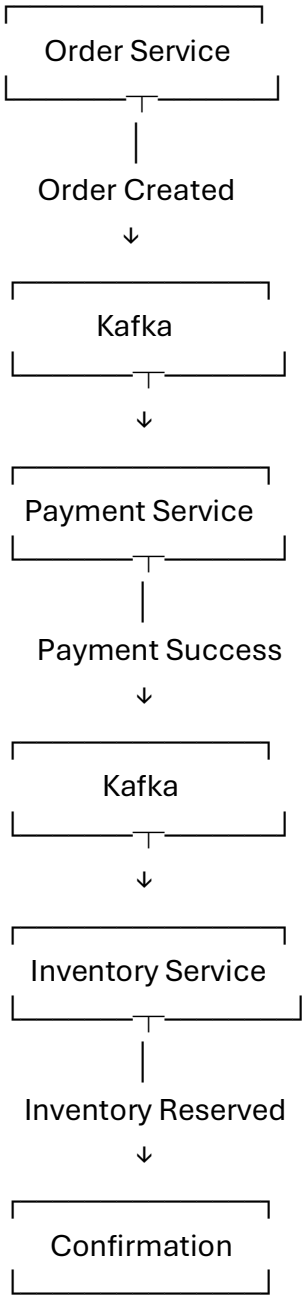
In code, the Payment Service can listen for the failure:

```
@KafkaListener(topics = "inventory-failed")
public void refundPayment(InventoryFailedEvent event) {

   // Refund payment

   kafkaTemplate.send(
      "payment-refunded",
      new PaymentRefundedEvent(event.getOrderId())
   );
}
```

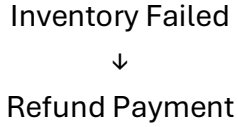
Then the Order Service can cancel the order:

```
@KafkaListener(topics = "payment-refunded")
public void cancelOrder(PaymentRefundedEvent event) {

   // Update order status

   orderRepository.updateStatus(
      event.getOrderId(),
      "CANCELLED"
   );
}
```

Step 8 — Final Architecture



Failure path



↓

Cancel Order

Task 03 is completed.

We have the Saga transaction flow, success scenario, failure scenario, and compensation/rollback logic.